

```
1  #include "Matrix3x3.h"
2  #include <cassert>
3
4
5
6
7  bool Matrix3x3::operator==(const Matrix3x3& aOther) const noexcept
8  {
9      const Matrix3x3& M = *this;
10
11      return M[0] == aOther[0] &&
12             M[1] == aOther[1] &&
13             M[2] == aOther[2];
14 }
15
16
17
18 Matrix3x3 Matrix3x3::operator*(const Matrix3x3& aOther) const noexcept
19 {
20     const Matrix3x3& M = *this;
21
22     const Vector3D lCol0 = aOther.column(0);
23     const Vector3D lCol1 = aOther.column(1);
24     const Vector3D lCol2 = aOther.column(2);
25
26
27     return Matrix3x3(
28         Vector3D(M[0].dot(lCol0), M[0].dot(lCol1), M[0].dot(lCol2)),
29         Vector3D(M[1].dot(lCol0), M[1].dot(lCol1), M[1].dot(lCol2)),
30         Vector3D(M[2].dot(lCol0), M[2].dot(lCol1), M[2].dot(lCol2))
31     );
32 }
33
34
35
36 Matrix3x3 Matrix3x3::transpose() const noexcept
37 {
38     const Matrix3x3& M = *this;
39
40     return Matrix3x3(
41         M.column(0),
42         M.column(1),
43         M.column(2)
44     );
45 }
46
47
48
49
```

```
50 float Matrix3x3::det() const noexcept
51 {
52     const Matrix3x3& M = *this;
53
54     return
55         M[0][0] * (M[1][1] * M[2][2] - M[1][2] * M[2][1])
56         - M[0][1] * (M[1][0] * M[2][2] - M[1][2] * M[2][0])
57         + M[0][2] * (M[1][0] * M[2][1] - M[1][1] * M[2][0]);
58
59 }
60
61
62
63 bool Matrix3x3::hasInverse() const noexcept
64 {
65     return det() != 0.0f;
66 }
67
68
69
70
71 Matrix3x3 Matrix3x3::inverse() const noexcept
72 {
73     assert(hasInverse());
74
75     const Matrix3x3& M = *this;
76     const float lDet = 1.0f / det();
77
78     return Matrix3x3(
79         Vector3D(
80             lDet * (M[1][1] * M[2][2] - M[1][2] * M[2][1]),
81             lDet * (M[0][2] * M[2][1] - M[0][1] * M[2][2]),
82             lDet * (M[0][1] * M[1][2] - M[0][2] * M[1][1])
83         ),
84         Vector3D(
85             lDet * (M[1][2] * M[2][0] - M[1][0] * M[2][2]),
86             lDet * (M[0][0] * M[2][2] - M[0][2] * M[2][0]),
87             lDet * (M[0][2] * M[1][0] - M[0][0] * M[1][2])
88         ),
89         Vector3D(
90             lDet * (M[1][0] * M[2][1] - M[1][1] * M[2][0]),
91             lDet * (M[0][1] * M[2][0] - M[0][0] * M[2][1]),
92             lDet * (M[0][0] * M[1][1] - M[0][1] * M[1][0])
93         )
94     );
95 }
96
97
98 std::ostream& operator<<(std::ostream& aOStream, const Matrix3x3& aMatrix)
```

```
99 {
100     aOStream << "["
101             << aMatrix[0].toString() << ", "
102             << aMatrix[0].toString() << ", "
103             << aMatrix[1].toString() << ", "
104             << aMatrix[2].toString()
105             << "];"
106
107     return aOStream;
108 }
```